

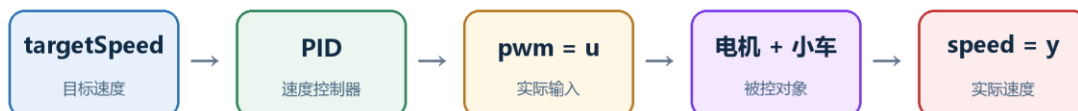
工程操作手册 · 版本 1.1

小车速度环 PID

自动辨识、调参与

复杂控制器优化说明

串口采集 → 本地/MATLAB 模型辨识 → PID/PSO 优化 → 参数回写



辨识阶段：绕过 PID，记录 pwm 与 speed；闭环阶段：PID 根据速度误差计算 pwm。

图1 speed、pwm 与 targetSpeed 在速度环中的关系

适用对象：单电机速度环、差速小车左右轮分通道整定、USB/蓝牙虚拟串口设备

2026-08-26 | PID Lab 上位机

先看结论：三个变量怎么接

核心映射 speed 是编码器测得的实际速度，也是系统输出 y ；pwm 是经过限幅后真正施加给电机的控制量，也是系统输入 u ；targetSpeed 是闭环控制目标，不等于辨识输入。

变量	含义	在辨识中的角色	建议单位/范围
targetSpeed	希望小车达到的速度	仅用于闭环验证，可作为 setpoint 上报	rpm、m/s 或编码器计数/s
speed	编码器计算出的实际速度	输出 y	必须与 targetSpeed 同单位
pwm	实际写入电机驱动的值	输入 u ，必须使用限幅后的真实值	推荐归一化 -1.0 ~ 1.0
Kp, Ki, Kd	PID 连续域参数	由本地搜索或 pidtune 整定，再部署到固件	与变量单位和 T_s 相关
T_s	固定控制/采样周期	本地模型和 MATLAB iddata 的时间基准	速度环可先用 0.01 s

闭环关系： $e = \text{targetSpeed} - \text{speed}$ ；PID 根据 e 计算 pwm；电机和小车响应后，编码器再次更新 speed。

辨识关系：暂时绕过 PID，用安全的阶跃、PRBS 或 chirp 直接生成 pwm；同时采集 t 、pwm 和 speed，交给本地 NumPy 或 MATLAB 后端拟合被控对象 $G(s)$ 。

最短可行流程

- 准备设备。** 架空驱动轮或使用滚筒台，设置独立急停和低 PWM 上限。
- 连接串口。** 在上位机中选择 COM 端口和 115200 波特率，关闭“模拟设备”。
- 采集辨识数据。** 先用 step、幅值 0.20 ~ 0.25、时长 8 s、采样周期 0.01 s。
- 辨识并整定。** 模型先选 1 个极点、0 个零点，控制器先选 PI。
- 检查再回写。** 确认拟合曲线、参数数量级和执行器安全状态后再写入。
- 小步验证。** 按 0→30→60→30→0 的目标速度序列试验，观察超调、振荡和饱和。

计算后端 默认的“本地 NumPy 自动调参”会从真实数据辨识 FOPDT 模型并搜索 PI/PID/PIDF，不需要 Codex、网络或 MATLAB。MATLAB Engine 是可选后端；“接口模拟”只验证界面与数据流，模拟结果禁止写入真实设备。

1 单片机程序应采用三种工作模式

速度环固件不要一直运行 PID。为了辨识，至少需要 STOP、IDENTIFY 和 PID 三种模式。协议解析与底层硬件解耦后，同一套上位机可以连接 STM32、MSPM0、ESP32、Arduino、51 或其他芯片。

模式	电机输入来源	上报数据	进入/退出条件
MODE_STOP	固定为 0	状态消息，可选	上电、stop、超时、故障
MODE_IDENTIFY	激励信号发生器	t、实际 pwm、speed	收到 start；到时或 stop 退出
MODE_PID	速度 PID 输出	t、pwm、speed、setpoint	参数有效并明确启用闭环

基础变量 (C 语言骨架)

```
typedef enum { MODE_STOP, MODE_IDENTIFY, MODE_PID } ControlMode;

volatile ControlMode mode = MODE_STOP;
float targetSpeed = 0.0f;
float speed = 0.0f;
float pwm = 0.0f;
float kp = 0.0f, ki = 0.0f, kd = 0.0f;
float integral = 0.0f, previousError = 0.0f;
const float Ts = 0.01f;
```

1.1 固定周期控制任务

使用硬件定时器每 T_s 秒运行一次控制任务。不要用不稳定的主循环延时替代固定周期；串口解析可以在主循环或通信线程中完成。

速度环核心逻辑

```

void control_tick(void) {
    speed = encoder_get_speed();

    if (mode == MODE_IDENTIFY) {
        pwm = clamp(generate_test_pwm(), -0.40f, 0.40f);
        motor_set_pwm(pwm);
        send_sample(experiment_time(), pwm, speed, 0.0f);
    } else if (mode == MODE_PID) {
        float e = targetSpeed - speed;
        float i_before = integral;
        integral += ki * Ts * e;
        float derivative = (e - previousError) / Ts;
        float raw = kp * e + integral + kd * derivative;
        pwm = clamp(raw, -1.0f, 1.0f);
        if (raw != pwm) integral = i_before; // basic anti-windup
        previousError = e;
        motor_set_pwm(pwm);
        send_sample(run_time(), pwm, speed, targetSpeed);
    } else {
        motor_set_pwm(0.0f);
    }
}

```

必须记录实际输入 sample.u 必须等于限幅、死区补偿和方向处理之后真正写给驱动器的 pwm；如果上报的是未限幅的 raw，任一辨识后端都会用错误输入建模。

1.2 start、stop 和 set_pid 的处理

- 收到 start：校验 signal、amplitude、duration 和 sample_time；清零实验计时；将模式切到 MODE_IDENTIFY。
- 收到 stop：立刻切到 MODE_STOP，pwm 置 0，并返回 ack。
- 收到 set_pid：再次校验参数范围，保存 Kp/Ki/Kd/N，清零积分项和历史误差；返回 ack。
- 收到 set_controller：按 algorithm、layer 和参数白名单逐项校验；先写临时结构，全部合法后在安全周期边界原子更新。
- 闭环启用：当前协议没有 setpoint/模式命令。可由固件按键或预设 targetSpeed 进入 MODE_PID；更推荐按第 6 章增加 set_setpoint 与 set_mode。
- 通信看门狗：若规定时间内没有合法命令或心跳，必须进入 MODE_STOP。

2 串口协议：不同芯片保持同一种上位机接口

传输层可以是 USB CDC、USB 转 UART 或蓝牙 SPP。只要操作系统把它枚举为 COM 口，上位机的处理方式相同。默认串口参数为 115200、8 数据位、无校验、1 停止位（8N1）。

帧格式 UTF-8 JSON Lines：一行一个完整 JSON 对象，以换行符 \n 结束。禁止把调试日志混在协议行中；调试信息请封装为 status 消息。

2.1 自动握手、设备档案和 request_id

连接后的能力握手

```
{
  "type": "hello", "protocol_version": 2,
  "request_id": "e4f62dc3ec5a45fb"
},
{
  "type": "capabilities", "request_id": "e4f62dc3ec5a45fb",
  "device_id": "cart-speed-loop-01", "firmware_version": "1.1.0",
  "protocol_version": 2,
  "supported_commands": ["hello", "start", "stop", "get_pid",
    "set_pid", "validate_pid", "accept_pid", "abort_validation"],
  "units": { "input": "pwm", "output": "rpm", "setpoint": "rpm" },
  "limits": { "excitation_abs_max": 0.35,
    "actuator_abs_max": 1.0, "setpoint_abs_max": 300.0,
    "sample_time_min": 0.005, "sample_time_max": 0.1 }
}
```

- device_id 必须对同一台设备稳定，不要用会变化的 COM 口号。上位机据此选中持久化档案。
- supported_commands 是固件已实现命令的白名单；units 用于界面标注；limits 用于上位机预检查。
- 每条命令都有唯一 request_id，设备的直接响应必须原样回传。迟到或串包 ACK 不会提交新的 PID 版本。
- 设备 1.5 s 内不回复 capabilities 时，软件进入旧固件兼容模式：基础采集与 PID 写入可用，自动闭环验收和复杂控制器写入禁用。
- 设备档案保存在 %LOCALAPPDATA%\PIDLab\device_profiles.json，连接区可编辑波特率、单位和安全范围。

安全边界 档案范围只是上位机的第一道预检查。固件仍必须重新校验所有参数，并独立实现执行器限幅、通信看门狗、传感器异常停机和硬件急停。

2.2 上位机下发

当前上位机已支持的命令

```
{
  "type": "start",
  "signal": "step",
  "amplitude": 0.25,
  "duration": 8.0,
  "sample_time": 0.01,
  "request_id": "69a6c4f9baba40e1"
}
{
  "type": "stop",
  "request_id": "31af96166be245c4"
}
{
  "type": "set_pid",
  "kp": 0.12,
  "ki": 0.85,
  "kd": 0.0,
  "n": 0.0,
  "request_id": "9f0aab8395664f56"
}
{
  "type": "set_controller",
  "algorithm": "cascade_ball_balance",
  "layer": "inner_velocity",
  "params": {
    "VELOCITY_KP_NORMAL": 0.04,
    "VELOCITY_KI_NORMAL": 0.0012
  },
  "request_id": "f30b3b9aef1a4116"
}
```

命令	关键字段	固件动作
start	signal / amplitude / duration / sample_time	进入辨识模式并按固定周期输出激励
stop	无	停止实验，输出清零
set_pid	kp / ki / kd / n	校验、保存参数并清空 PID 状态
set_controller	algorithm / layer / params	校验参数白名单，原子更新复杂控制器当前层

2.3 设备上报

当前已支持的消息

```
{
  "type": "sample",
  "t": 0.51,
  "u": 0.25,
  "y": 32.6,
  "setpoint": 0.0,
  "request_id": "69a6c4f9baba40e1"
}
{
  "type": "ack",
  "command": "start",
  "ok": true,
  "request_id": "69a6c4f9baba40e1"
}
{
  "type": "status",
  "message": "device ready"
}
```

字段	要求	常见错误
t	从本次实验开始计时，单位秒，严格递增	使用系统绝对时间、重复时间戳
u	真实施加给对象的 pwm	上报理论命令而不是限幅后输出
y	编码器测得的 speed	与 targetSpeed 单位不同、方向符号相反
setpoint	可选；闭环显示目标速度	辨识时误把它当成 u

2.4 移植到不同芯片时只替换硬件层

- encoder_get_speed(): 把定时窗口内的编码器计数换算成 rpm、m/s 或计数/s。

- `motor_set_pwm()`: 处理方向引脚、PWM 映射、死区补偿和最终限幅。
- `uart_read_line()/uart_write_line()`: 实现一行一帧，并限制最大帧长。
- `control_tick()`: 由硬件定时器稳定调用；协议中的 `sample_time` 必须与实际 T_s 一致。
- `safety_stop()`: 过流、失速、通信超时、急停或异常参数时无条件清零输出。

3 上位机操作：从采样到参数回写



默认本地 NumPy 真实计算；合法 MATLAB 与所需工具箱是可选后端。

图2 PID Lab 的数据与计算流程

1. **连接设备。**选择 COM 端口、115200 波特率并连接。确认顶部显示正确的设备 ID、固件版本、单位和能力；实车时不要勾选“模拟设备”。
2. **设置激励。**首次试验选择 step，幅值 0.20 ~ 0.25，时长 8 s，采样周期 0.01 s。
3. **开始采集。**上位机发送 start；固件进入 MODE_IDENTIFY，持续上报 t/u/y。
4. **保存完整会话。**停止后点击“保存会话”生成 .pidlab 文件；它会同时归档原始数据、设备与实验设置、辨识结果、PID、效果评分和复杂控制结果。CSV 只用于和其他软件交换原始数据。
5. **选择模型。**初次使用 1 极点、0 零点；速度响应有明显二阶特征时再尝试 2 极点。
6. **选择控制器。**编码器速度环先选 PI；只有存在明确需求且速度滤波充分时再试 PID/PIDF。
7. **执行真实计算。**默认选择“本地 NumPy 自动调参”；已安装合法 MATLAB 和所需工具箱时，也可选 MATLAB Engine。
8. **审查结果。**查看拟合曲线、fit%、传递函数和 $K_p/K_i/K_d$ ；异常时不要回写。
9. **安全回写。**确保执行器可急停、目标速度为低值，再点击“写入 PID 到设备”。

3.1 第一次速度环试验的推荐值

项目	建议起点	调整依据
激励类型	step	容易确认方向和稳态增益；之后用 PRBS/chirp 丰富频段
PWM 幅值	0.20 ~ 0.25（归一化）	能克服死区但不过流；空载先更低
时长	8 s	至少覆盖 4 ~ 6 个主要时间常数
采样周期 T_s	0.01 s	稳定且显著快于机械响应
传递函数	1 极点、0 零点	拟合不足再增加阶次，避免盲目高阶
控制器	PI	微分项会放大编码器量化噪声

3.2 数据质量门槛

- 程序硬性要求至少 20 个点；工程上建议一次实验取得数百个稳定采样点。
- 时间戳必须严格递增；采样周期最大抖动超过中位周期的 20% 时，当前程序拒绝辨识。
- u 必须有足够变化， y 必须产生可辨识响应；若电机停在死区内，增大幅值或加安全偏置。
- 采样期间不应出现长时间丢包、手动干预、机械碰撞或电源电压明显下跌。

3.3 保存、恢复和对比实验会话

1. **保存会话。**当前有采样数据时，点击“保存会话”并选择文件名。生成的 .pidlab 是可移植压缩包，内部包含带校验和的 samples.csv 和 manifest.json。
2. **恢复会话。**点击“载入会话”即可恢复波形、设备元数据、实验参数、传递函数、PID、效果分析和复杂控制优化结果；旧的 csv 导入仍然可用，但只恢复 $t/u/y/setpoint$ 。
3. **对比两次实验。**先载入或采集当前实验，再点击“与会话对比”选择一个基准文件。软件会叠加基准输出曲线，并比较拟合度、闭环评分、超调、调节时间、稳态误差、饱和率和 PID 参数。
4. **检查设备匹配。**会话记录稳定 device_id；当载入会话所属设备与当前连接设备不一致时，软件允许离线查看和重新分析，但禁止把其中参数写入当前设备。

为什么仍保留 CSV CSV 适合导入 MATLAB、Python、Excel 或 PlotJuggler；.pidlab 适合完整复现实验。建议每次正式调参都保存 .pidlab，需要外部分析时再额外导出 CSV。

3.4 模拟演示：看本地与 MATLAB 的差异

顶部“模拟演示”仍用于快速跑通单一工况；“对比演示”会对同一组模拟采样依次运行本地 NumPy 和 MATLAB Engine，并在“频域 / 稳定性”页画出单位阶跃闭环响应。辨识实验中的“模拟场景”可以切换标准二阶、快速对象、慢响应、强测量噪声以及输入限幅+死区，用于观察算法对工况变化的敏感性。

对比输出	怎么看	注意
闭环曲线	目标、本地 NumPy、MATLAB 三条曲线叠加	同一模型下比较趋势，不能替代实车
超调/调节时间	越小不一定越好，要兼顾噪声和执行器余量	以低幅验证门限为准
稳态误差	检查积分作用与模型静态增益	死区、摩擦和饱和会造成偏差
PM/GM/Ms	在频域页查看相位裕度、增益裕度和灵敏度峰值	门限不通过时写入按钮会锁定

为什么 MATLAB 可能看起来更差 高阶 tfest 在单一阶跃激励下可能出现过拟合或稳态方向错误。工程入口会比较实测稳态方向；如果高阶模型的 DC 增益符号相反且低一阶模型拟合更好，会自动采用低一阶模型并在结果中说明。仍建议用 PRBS/chirp 丰富频段，再决定最终模型阶次。

4 上位机如何得到模型和 PID 参数

默认本地后端用 NumPy 从阶跃数据估计一阶惯性加纯延迟（FOPDT）模型，再按保守、均衡或快速策略搜索 PI/PID/PIDF，并用闭环仿真的超调、稳态误差和饱和率做写入门禁。这条路径不依赖 Codex、网络或 MATLAB。

可选 MATLAB 后端 安装了合法 MATLAB、System Identification Toolbox 和 Control System Toolbox 时，也可调用 tfest、compare 和 pidtune。tfest 是数值系统辨识算法，不是生成式 AI。

工程内 MATLAB 计算链

```
rawData = iddata(y, u, [], "SamplingInstants", t);
data = detrend(rawData, 0);
plant = tfest(data, poles, zeros, tfestOptions(...));
[controller, ~] = pidtune(plant, controllerType);
[compared, fit] = compare(data, plant);
```

阶段	函数	输出/判定
数据封装	iddata(y, u, [], SamplingInstants)	保留采样时刻并估计连续时间 G(s)
去偏置	detrend(..., 0)	减少直流偏置对模型的影响
模型辨识	tfest + 方向检查	得到稳定连续传递函数 G(s)，必要时回退到较低阶次
模型验证	compare	得到拟合曲线和 fitPercent
控制器整定	pidtune	得到 Kp、Ki、Kd；PIDF 还得到滤波参数 N

模型可写成 $G(s)=Y(s)/U(s)$ 。速度环常见的一阶近似为 $G(s)=K/(ts+1)$ ，但最终阶次应由数据、残差和闭环效果共同决定。

4.1 如何看拟合度

经验解释 fitPercent 越高通常表示模型越能重现这段数据，但高拟合度不保证闭环一定安全。可把 80% 以上视为较好起点，60% ~ 80% 需要结合曲线检查，明显低于 60% 时优先重做实验；这些不是硬性标准。

- 模型曲线应同时跟随上升过程、稳态值和主要滞后，不能只在局部重合。
- 若增加模型阶次只让 fit 略升，却出现不合理极点/零点，保留较简单模型。
- 同一对象换一组激励重新辨识，参数应处在相近数量级；差异很大说明对象非线性、数据不足或工况变化。
- 本地 NumPy 后端可以真实计算而不需 MATLAB；选择 MATLAB 后端时才必须能启动 MATLAB 并取得相应工具箱许可证。不要把“接口模拟”参数当成辨识结果。

4.2 为什么速度环先用 PI

速度编码器有量化噪声，微分会放大高频变化。PI 已能消除大多数稳态误差，调试风险也更低。只有在 PI 无法满足动态性能、测速滤波充分且采样周期稳定时，才考虑 PID/PIDF。

5 整定参数如何放进基础 PID 程序

5.1 写法 A：显式使用采样周期

如果固件按下面方式实现积分和微分，上位机本地后端或 MATLAB 后端输出的连续域 K_p 、 K_i 、 K_d 可以直接使用；前提是 speed、targetSpeed、pwm 的单位与辨识时完全一致。

推荐实现

```
e = targetSpeed - speed;
integral += Ki * Ts * e;
derivative = (e - previousError) / Ts;
raw = Kp * e + integral + Kd * derivative;
pwm = clamp(raw, PWM_MIN, PWM_MAX);
```

5.2 写法 B：积分只累加误差，微分只做差

如果基础代码写成 $I += e$ 、 $D = e - e_{prev}$ ，那么固件中的积分/微分系数必须换算。不要把上位机输出的连续域 K_i 、 K_d 原样填进这种离散写法。

离散系数换算

```
Ki_discrete = Ki_matlab * Ts;
Kd_discrete = Kd_matlab / Ts;

I += e;
D = e - previousError;
raw = Kp_matlab * e + Ki_discrete * I + Kd_discrete * D;
```

项目	建议处理	原因
输出限幅	始终对 pwm 限幅	防止过流和失控；辨识与闭环要用同一标度
抗积分饱和	饱和时冻结积分或做回算	避免解除饱和后长时间反向超调
模式切换	清零 integral 与 previousError	避免旧状态造成输出突跳
微分滤波 N	PI 时忽略；PIDF 时按同一结构实现	否则 MCU 行为与上位机控制器模型不一致
方向符号	正 pwm 应产生正 speed	符号相反会形成正反馈

单位一致性 如果辨识时 u 用 $0 \sim 1$ 归一化 pwm ，而闭环代码用 $0 \sim 1000$ 计数，则参数必须按输入比例换算；最稳妥的做法是固件内部始终使用 $-1.0 \sim 1.0$ ，再在 `motor_set_pwm()` 内映射到定时器计数。

5.3 写入后如何启用闭环

当前上位机能够下发 `set_pid`，但尚未提供目标速度输入命令。实际使用可先在固件里设置低速 `targetSpeed` 并由实体按键进入 `MODE_PID`；长期方案见下一章的协议扩展。不要让收到 `set_pid` 自动等同于电机立即启动。

6 建议增加的闭环控制协议

当前状态 以下 `set_setpoint`、`set_mode`、`offset/minimum/maximum` 是建议扩展，不是当前上位机已经实现的字段；上位机和固件需要同时增加解析与校验。

6.1 目标速度和模式命令

建议扩展命令

```
{
  "type": "set_setpoint", "value": 60.0, "unit": "rpm",
  "request_id": "4a820b5e9a1e40cb"
}
{
  "type": "set_mode", "mode": "pid",
  "request_id": "dc46e96fd4d1477e"
}
{
  "type": "set_mode", "mode": "stop",
  "request_id": "6b622736ed074dcb"
}
```

- `set_setpoint` 只更新 `targetSpeed`，不应绕过速度/加速度限幅。
- `set_mode=pid` 前检查 PID 参数已验证、编码器有效、急停未触发且目标速度在安全范围。
- `set_mode=stop` 的优先级必须最高，并立即清零执行器。
- 建议设备回复 `ack`，并通过 `status` 上报当前模式和故障原因。

6.2 只允许正转的小车如何做 PRBS/chirp

双极性 PRBS 会产生负 pwm ，不适合只能前进或传动有明显反向间隙的机构。可在安全工作点附近增加偏置：实际 $pwm = \text{clamp}(\text{offset} + \text{excitation}, \text{minimum}, \text{maximum})$ 。

建议的偏置激励扩展

```
{"type": "start", "signal": "prbs", "offset": 0.30,
  "amplitude": 0.05, "minimum": 0.0, "maximum": 0.45,
  "duration": 8.0, "sample_time": 0.01,
  "request_id": "9e5d8027eec84540"}
```

这个示例让 pwm 主要在 0.25 ~ 0.35 之间变化，同时硬限制在 0 ~ 0.45。sample.u 仍然必须上报最终实际 pwm。

6.3 差速小车的左右轮

当前计算链是单输入单输出（SISO），一次只辨识一个速度环。差速小车应分别固定或隔离另一侧，独立采集左轮和右轮，得到两组参数 $K_{pL}/K_{iL}/K_{dL}$ 与 $K_{pR}/K_{iR}/K_{dR}$ 。

- 先对左右轮使用相同单位、相同 T_s 和相近激励幅值，便于比较。
- 若两侧机械差异明显，不要强行共用一组参数。
- 底盘上层速度/转向控制应向左右速度环发送不同 targetSpeed；底层环仍各自使用实际 speed。
- 后续可在协议中增加 channel 字段，但当前上位机需要相应改造后才能同时显示多通道。

7 复杂串级控制器与可扩展参数优化

不是只有三个参数 K230 示例不是单一 PID，而是状态估计、位置外环、速度内环、增益调度、制动规划、前馈/阻尼和执行器约束的组合。上位机现在用参数注册表描述算法，用 PSO 只优化勾选项；以后增加新算法或新参数时不必改串口帧结构。

7.1 K230 控制核心的实际结构



图3 K230 风格串级控制器的模型化结构

- 状态估计：三状态 Kalman 估计位置、速度和加速度，为控制器提供平滑反馈。
- 位置外环：近区/远区使用不同 PD 增益，并根据剩余距离和制动加速度限制目标速度。
- 速度内环： $K_p/K_i/K_d$ 随位置与速度误差变化，带微分低通、积分泄漏和条件抗饱和。
- 补偿项：速度给定前馈、球加速度阻尼、位置倾角助推共同修正目标角度。
- 执行器整形：目标角低通、单周期角度步长和软角度限位之后才换算为电机脉冲。

7.2 参数按层分组，不建议一次全部放开

优化层	上位机标识	典型参数	推荐顺序
速度内环	inner_velocity	速度 $K_p/K_i/K_d$ 的近区、常规、扰动值；微分滤波时间常数	先独立辨识角度到速度模型，再优化
位置外环	outer_position	位置 K_p/K_d 、目标速度上限、制动加速度和制动裕量	固定已验证内环后优化
补偿与约束	compensation	前馈、加速度阻尼、位置助推、角度低通、步长、软限位	最后逐项小范围优化
联合核心	joint_core	从前三组选择少量强相关核心参数	只在各层稳定后使用

维数控制 联合页会显示全部注册参数，但默认只勾选少量核心项。PSO 的搜索量随参数维数明显上升；一次放开二十多个参数既慢，也更容易得到难以实机解释的组合。

7.3 在上位机中怎样使用

1. **采集对应数据。**内环应记录舵机/轨道角度 u 与球速度 y ；外环或整体模型可记录角度 u 与球位置 y 。
2. **先完成真实辨识。**在“标准 PID”页用本地 NumPy 或 MATLAB 得到可部署模型；接口模拟结果只能验证数据流。
3. **选择模板和层级。**打开“复杂控制器 / PSO”，选择 K230 串级球控以及速度内环、位置外环、补偿或联合核心。
4. **声明模型输出。**按采集时 y 的含义选择“速度”或“位置”；选择错误会让仿真目标和反馈变量不一致。
5. **设置搜索范围。**为每个参数填写初值、下限和上限，只勾选本轮真正要优化的参数。
6. **运行并审查。**检查目标函数改善、归一化 MAE、超调、调节时间、饱和占比以及参考/响应曲线。
7. **低幅写入验证。**只有通过质量与安全门禁的真实辨识结果可写入；固件支持 `set_controller` 后，在限幅和急停条件下分层试车。

7.4 PSO 优化的目标与边界

当前优化器用 NumPy 实现粒子群算法，不依赖额外 PSO 软件包。每个粒子代表一组勾选参数；初始参数会作为候选粒子保留，因此搜索结果不会故意丢弃当前基线。

当前目标函数（权重可在 `optimizer.py` 中扩展）

```
J = 2.2*MAE_norm + 1.0*RMSE_norm + 3.0*overshoot
  + 0.35*settling_norm + 0.18*control_energy
  + 1.5*saturation_ratio + 0.08*control_variation
  + parameter_consistency_penalty
```

上位机在辨识传递函数上用连续状态空间和 RK4 仿真控制核心。它会处罚误差、超调、长调节时间、控制能量、长期饱和、指令跳变以及近/远参数次序不合理。

模型边界 K230 程序中的任务状态机、终点锁存、静差恢复、视觉丢失处理、摩擦/间隙和舵机死区未全部进入当前优化模型。PSO 结果是更系统的候选初值，不是无人监护下直接部署的保证。

7.5 新参数怎样通过串口传给固件

复杂控制器参数消息

```
{
  "type": "set_controller",
  "algorithm": "cascade_ball_balance",
  "layer": "outer_position",
  "params": {
    "POSITION_KP_NEAR": 1.08,
    "POSITION_KP_FAR": 2.42,
    "POSITION_BRAKE_MARGIN": 0.79
  },
  "request_id": "7bf59692453847b3"
}
```

- algorithm 是控制器模板标识，layer 表示本次更新的层；params 可以只包含勾选并优化后的参数。
- 固件必须使用参数白名单和独立安全范围，拒绝未知算法、未知层、未知参数、NaN、无穷和越界值。
- 先把所有字段写入临时结构，只有全部合法时才一次性替换当前参数，避免半更新。
- 更新在安全控制周期边界生效；改变 PID/滤波结构时清空对应积分器、微分器和滤波状态。
- 设备回复 ack，command 必须为 set_controller；拒绝时在 message 中说明原因。

7.6 增加另一种优化算法或新的控制参数

参数数量不再固定为三个。新增控制器时，在 pidlab/controllers.py 注册 ControllerSchema 和 ParameterSpec；界面据此自动生成参数行，协议仍使用 set_controller。然后在 pidlab/optimizer.py 增加该算法的仿真控制律，并在固件中增加同名 algorithm 的白名单与应用函数。

改动位置	职责	通常无需改动
controllers.py	声明参数名、中文标签、分组、默认值、上下限和单位	串口读写线程
optimizer.py	实现控制律仿真、目标函数或新的优化器	JSON Lines 帧结构
固件参数表	按 algorithm/layer 校验并原子应用	上位机 COM/蓝牙连接层
测试	覆盖参数范围、消息往返和优化器基线	标准 set_pid 兼容路径

8 上车验证与安全验收

8.1 逐级验证顺序

1. **空载方向检查。**给很小的正 pwm，确认 speed 为正；反向也应一致。
2. **低幅辨识。**在轮子架空或滚筒台上完成低幅 step，确认无过流、无打滑。
3. **模型复核。**检查拟合曲线和传递函数数量级，使用另一组数据交叉验证。
4. **低目标闭环。**先给 20% ~ 30% 额定速度，观察 pwm 是否长时间饱和。
5. **阶梯目标。**按 0→30→60→30→0（示例单位 rpm）测试跟踪。
6. **负载测试。**落地后逐步增加目标与负载，记录电流、温升、超调和稳态误差。

8.2 建议验收指标

下列数值适合作为首次调试目标，不是所有小车的强制标准；最终应按机械、电机、电源和比赛/产品需求确定。

观察项	首次调试目标	不合格时优先动作
方向	正目标产生正速度，负目标产生负速度	修正编码器或电机方向符号
稳定性	无持续振荡、无发散	降低 K_p/K_i ，检查 T_s 和符号
超调	可先控制在约 10% ~ 15% 内	降低 K_p/K_i 或放缓目标斜率
稳态误差	稳定工况尽量小于目标的 5%	检查积分、死区和负载变化
饱和	除启动瞬间外不应长期顶住上限	降低目标/增益，检查电机能力
温升与电流	始终低于硬件允许值	立即停机并降低输出限制

急停原则 上位机的“停止”按钮不能作为唯一保护。高功率设备必须有独立硬件急停；任何过流、编码器异常、通信超时、软件看门狗故障都应在下位机本地清零 PWM。

8.3 快速故障排查

现象	最可能原因	处理
COM 口看不到	驱动未装、蓝牙未建立 SPP、端口被占用	检查设备管理器，关闭占用串口的软件
有 u 没有 y	编码器读取/单位换算错误，PWM 未克服死区	单独验证测速；缓慢提高安全幅值
时间戳或抖动报错	控制任务不固定、串口阻塞	定时器采样；通信与控制解耦
本地辨识/tfest 失败	输入变化不足、阶次不当、数据含碰撞/饱和	重采数据，本地后端先用标准阶跃；MATLAB 从 1 极点 0 零点开始
闭环持续振荡	Kp/Ki 过大、方向符号错、Ts 配置错	立即停机，先核对符号和周期，再降增益
响应很慢	Kp/Ki 太小、目标限幅过严、动力不足	逐步提高增益；检查电源、电流和负载
解除饱和后仍冲	积分饱和	加入冻结/回算抗饱和并在切换时清积分
MATLAB 无法真实计算	Engine、许可证或工具箱不可用	检查 MATLAB/许可证；接口模拟仅用于开发

附录 A 单片机实现检查清单

- speed 与 targetSpeed 使用同一单位，正负方向一致。
- pwm 采用统一标度，并上报限幅后的实际值。
- 控制任务由硬件定时器固定周期执行，Ts 与协议一致。
- STOP / IDENTIFY / PID 三种模式互斥，故障时无条件进入 STOP。
- 实现 hello/capabilities，声明稳定 device_id、固件版本、命令白名单、单位和安全范围。
- 每条直接响应原样回传 request_id；实验/验证完成与中止消息使用对应会话 ID。
- 解析 start、stop、set_pid；复杂控制器还要按白名单解析 set_controller。
- 通信超时、过流、编码器异常和急停均能本地关闭执行器。
- PID 有输出限幅、抗积分饱和和模式切换状态清零。
- 复杂参数先写临时结构，全部合法后在安全周期边界原子更新。
- 先用 PI 低速验证，再逐步提高工况；模拟结果不部署到真实设备。

附录 B 工程内相关文件

文件	用途
README.md	上位机安装、启动和总体流程
docs/device_protocol.md	当前 JSON Lines 设备协议
examples/arduino_reference/arduino_reference.ino	Arduino 风格串口与激励参考固件
matlab/pidlab_identify_and_tune.m	iddata → tfest → compare → pidtune 入口
pidlab/protocol.py	消息编码、解码与 sample 数据结构
pidlab/device_profiles.py	设备能力解析、安全范围校验与档案持久化
pidlab/matlab_backend.py	数据校验、MATLAB Engine 调用与接口模拟
pidlab/frequency_analysis.py	Bode、闭环极点、裕度与灵敏度峰值安全门禁
pidlab/demo.py	本地 NumPy / MATLAB 同数据对比及单位阶跃响应
pidlab/simulator.py	多种模拟对象工况和内置模拟设备
pidlab/local_autotune.py	无 MATLAB 的 FOPDT 辨识、PID 搜索与写入安全门禁
pidlab/controllers.py	控制器模板、参数分组、默认值、安全范围和 set_controller 构造
pidlab/optimizer.py	串级控制核心仿真、性能指标与 NumPy PSO 优化
pidlab/sessions.py	完整 .pidlab 实验会话的保存、校验、恢复和指标对比
tests/test_controllers.py	参数注册表和通用控制器消息测试
tests/test_optimizer.py	传递函数仿真、指标和 PSO 基线测试
tests/test_sessions.py	会话往返、损坏检测、未知指标和实验对比测试
tests/test_frequency_analysis.py	频域裕度、稳定性和安全门限测试
tests/test_demo.py	模拟闭环响应和算法对比测试

附录 C GitHub 架构参考

以下项目用于比较工作流和软件结构。本工程没有直接运行其安装器或复制未知二进制；算法参数、协议和安全边界仍以当前工程为准。

- **System_Identification**: [GitHub 项目主页](#)。MATLAB 系统辨识与串级速度/位置环思路，支持先整定速度环、再处理位置环的顺序。
- **pyPIDTune**: [GitHub 项目主页](#)。过程反应曲线采集、PID 整定、仿真和设备模拟的完整工具链参考。
- **Ball-Balancing-PID-System**: [GitHub 项目主页](#)。双 PID、串口通信、视觉与执行器解耦的模块化球平衡结构参考。
- **aerial_balance_bench**: [GitHub 项目主页](#)。串级 PID、状态预测、模型控制以及仿真到实机验证的组织方式参考。
- **pyswarm**: [GitHub 项目主页](#)。带约束 PSO 接口设计参考；本工程使用自己的轻量 NumPy 实现，不增加该运行时依赖。
- **python-control**: [GitHub 项目主页](#)。开源 SISO/MIMO 控制系统分析库；本工程用它计算反馈、频率响应、稳定裕度和闭环极点。

下一步 先用当前固件采集与所选层级相匹配的 u/y 数据，用本地 NumPy 或 MATLAB 完成真实辨识；再从速度内环的 2 ~ 4 个参数开始小范围 PSO。固件实现 set_controller 白名单和原子更新后，才在低幅、可急停条件下回写验证。